Tokens firmados frente a sesiones de servidor: amenazas, revocación y almacenamiento seguro en aplicaciones web empresariales

María Stephanie Vargas Ramírez
Escuela de Informática y Computación, Universidad Nacional de Costa Rica
maría.vargas.ra@est.una.ac.cr
EIF509 Desarrollo de Aplicaciones Web · II Ciclo 2026 · Prof. Elberth Adrián Garro Sánchez

***Resumen*—La autenticación web no termina al verificar credenciales: la credencial de sesión debe protegerse durante todo su ciclo de vida. Este artículo compara las sesiones mantenidas en el servidor con los tokens firmados en formato JSON Web Token (JWT) bajo un modelo de amenazas explícito, que incluye la ejecución de código inyectado (XSS, cross-site scripting), la falsificación de solicitudes entre sitios (CSRF), el robo y la reutilización de credenciales y el uso de tokens después de cerrar sesión o de cambiar privilegios. La comparación se organiza en cinco criterios técnicos —seguridad, rendimiento, complejidad, costo y mantenibilidad— y se contrasta con estándares del IETF, guías de OWASP y las directrices de identidad digital del NIST. Los hallazgos indican que un JWT no es más seguro por sí mismo: su ventaja es la validación local sin consultar un almacén central y su costo principal es la dificultad de revocarlo antes de que expire; las sesiones de servidor permiten invalidación inmediata, pero exigen un almacén compartido y controles CSRF cuando viajan en cookies. Guardar credenciales en localStorage amplía el impacto de XSS y debe evitarse. Para un sistema web empresarial con reglas de negocio y una arquitectura de backend único, se decide adoptar sesión de servidor con cookie HttpOnly, Secure y SameSite, y reservar JWT de vida breve con rotación de refresh tokens para escenarios distribuidos.**

***Palabras clave*—autenticación web, tokens firmados, sesiones de servidor, revocación, seguridad de cookies**

I. INTRODUCCIÓN

Las aplicaciones web empresariales deben mantener la identidad autenticada entre solicitudes HTTP, un protocolo que por diseño no conserva estado, sin exponer credenciales reutilizables. La credencial que el navegador presenta en cada solicitud funciona como una credencial portadora (*bearer*): quien la posee puede actuar como el usuario legítimo mientras sea aceptada [1], [2]. Por eso la decisión entre una sesión de servidor y un JSON Web Token (JWT) [3] no es solo de formato: determina dónde reside el estado, cómo se valida cada petición y qué tan rápido puede cortarse el acceso cuando una cuenta se compromete.

En la práctica reciente, los JWT se han popularizado en interfaces de programación (API) y aplicaciones de página única bajo la idea de que ser «sin estado» los hace más escalables y modernos. Esa idea suele mezclar tres decisiones independientes: el formato de la credencial (identificador opaco o token autocontenido), su transporte (cookie o cabecera) y su almacenamiento en el navegador (memoria, *localStorage* o cookie). Confundirlas produce una falsa sensación de seguridad: un JWT firmado sigue siendo válido para quien lo robe, y una cookie protegida no elimina por sí sola todos los riesgos [1], [4].

El objetivo de este trabajo es comparar ambas alternativas bajo un modelo de amenazas explícito y decidir cuál aplicar al sistema que el equipo desarrolla en el laboratorio. Las preguntas guía son: (1) ¿cómo se comparan según criterios de seguridad, rendimiento, complejidad, costo y mantenibilidad?; (2) ¿dónde debe almacenarse la credencial en el navegador?; y (3) ¿qué reglas de validación deben aplicarse en los procesos principales? La posición defendida es que la elección debe depender de la sensibilidad de los datos, la necesidad de revocación, la arquitectura y los controles del navegador, no de una preferencia general por soluciones sin estado. La metodología es un análisis documental de fuentes normativas y técnicas primarias; no se realizaron mediciones propias. El artículo se organiza así: la sección II presenta el marco conceptual; la III, el análisis comparativo; la IV, la aplicación al dominio propio; y la V, las conclusiones.

II. MARCO CONCEPTUAL

A. Sesiones de servidor y cookies

En una sesión tradicional, el servidor genera un identificador aleatorio e impredecible y conserva en un almacén (memoria, base de datos o caché compartida) el contexto asociado: usuario, vencimiento absoluto y por inactividad, permisos y otros atributos. El navegador solo recibe el identificador; en cada solicitud, el servidor lo busca y decide si la acción se autoriza. Según OWASP, el identificador debe ser largo, aleatorio y sin significado, regenerarse tras autenticar o cambiar de privilegio, transmitirse solo por canales cifrados y destruirse en el servidor al cerrar la sesión [1]. Las directrices del NIST añaden requisitos de expiración, reautenticación y terminación de sesión que varían según el nivel de aseguramiento de la autenticación [2].

El transporte habitual es la cookie HTTP. El estándar original, RFC 6265, definió los atributos *Secure*, *HttpOnly*, *Domain*, *Path* y de vigencia [5]. La revisión que lo reemplaza, actualmente en la etapa final de publicación, incorpora *SameSite* (valores *Strict*, *Lax* y *None*), que restringe el envío de la cookie en solicitudes entre sitios, y los prefijos *__Secure-* y *__Host-*, que obligan al navegador a aplicar garantías adicionales; también limita el tamaño de nombre y valor a 4096 octetos [6].

B. JSON Web Token

Un JWT es un formato compacto de afirmaciones (*claims*) con tres partes: cabecera, contenido y firma. Define afirmaciones registradas como emisor (*iss*), sujeto (*sub*), audiencia (*aud*), expiración (*exp*), válido-desde (*nbf*) e identificador (*jti*) [3]. La firma permite detectar modificaciones, pero no oculta el contenido; la confidencialidad requiere cifrado. Las buenas prácticas del

IETF recogen amenazas conocidas: aceptar el algoritmo *none*, confundir algoritmos simétricos y asimétricos, validar de forma incompleta el emisor y la audiencia, y confiar en parámetros de cabecera controlados por quien presenta el token. En consecuencia, el servidor debe fijar una lista de algoritmos permitidos y verificar firma y *claims* antes de confiar en el contenido [4].

C. Modelo de amenazas

Se consideran siete capacidades del atacante: (A1) ejecutar JavaScript inyectado en la página (XSS) [7]; (A2) inducir al usuario autenticado a enviar una solicitud no deseada desde otro sitio (CSRF) [8]; (A3) obtener una copia de la credencial por malware, equipo compartido, registros o copias de respaldo; (A4) interceptar tráfico sin cifrar; y (A5) aprovechar que la credencial sigue vigente después de cerrar sesión, cambiar la contraseña o modificar los permisos del usuario. (A7) fijar un identificador de sesión conocido por el atacante antes de que la víctima se autentique [1]. Para los JWT se añade (A6) presentar un token manipulado que el servidor valide de forma incorrecta. Se asume que todo el tráfico usa HTTPS y se excluyen el compromiso del servidor y de la clave de firma. El modelo no supone que HTTPS corrija XSS, que una firma impida el robo del token, ni que ocultar la credencial a JavaScript elimine CSRF: cada defensa debe responder a una amenaza concreta.

D. Almacenamiento en el navegador

localStorage es accesible para cualquier script del mismo origen, de modo que un XSS puede leer y exfiltrar la credencial, y el dato persiste entre sesiones del navegador. Mantener el token solo en memoria evita la persistencia, pero se pierde al recargar la página y no impide que el script inyectado use la credencial desde la propia página. Una cookie *HttpOnly* no es accesible mediante *document.cookie*, lo que dificulta el robo directo del valor; sin embargo, el navegador la adjunta automáticamente a las solicitudes coincidentes, por lo que reaparece el riesgo CSRF, y un script inyectado todavía puede ejecutar acciones desde el navegador de la víctima [7], [8].

E. Tokens de acceso y de refresco

En arquitecturas con OAuth 2.0, el IETF recomienda tokens de acceso de vida corta y, para clientes públicos, que los tokens de refresco estén restringidos al emisor o se roten con detección de reutilización [9], [10]. El mecanismo DPoP vincula el token a una clave criptográfica del cliente, de modo que una copia robada no pueda reutilizarse sin esa clave [11]. Por su parte, la versión 5.0 del estándar de verificación ASVS de OWASP dedica capítulos específicos a la gestión de sesiones y a los tokens autocontenidos, lo que ofrece requisitos verificables para evaluar ambas alternativas [12].

III. ANÁLISIS COMPARATIVO

La comparación usa cinco criterios técnicos explícitos: (C1) seguridad, desglosada en exposición ante XSS, resistencia a CSRF, revocación e impacto de una filtración; (C2) rendimiento y escalabilidad; (C3) complejidad de implementación; (C4) costo operativo; y (C5) mantenibilidad y madurez del ecosistema. La Tabla I resume el contraste; el texto lo sustenta con las fuentes. No existe una alternativa universalmente segura: la implementación y el modelo de amenazas son determinantes.

TABLA I
COMPARACIÓN DE ALTERNATIVAS SEGÚN CRITERIOS TÉCNICOS

Criterio	Sesión de servidor + cookie	JWT autocontenido
C1 Revocación	Inmediata al invalidar el registro central.	Difícil antes de *exp* sin lista de revocación o estado adicional.
C1 XSS	*HttpOnly* evita leer la cookie; el script aún puede actuar en el navegador.	En *localStorage*, el XSS extrae y reutiliza el token.
C1 CSRF	Cookie automática: requiere *SameSite* y defensa CSRF.	En cabecera no hay CSRF clásico; en cookie sí.
C1 Filtración	La copia permite suplantar hasta invalidar o expirar.	La copia permite suplantar hasta *exp* o revocación.
C2 Rendimiento	Una consulta al almacén por solicitud.	Validación local; vuelve la consulta si se revoca.
C3 Complejidad	Baja con soporte del marco.	Claves, algoritmos, *claims*, renovación.
C4 Costo	Operar el almacén de sesiones.	Gestión de claves y de revocación.
C5 Mantenibilidad	Madura; menos código propio.	Permisos en *claims* pueden desactualizarse.

A. Seguridad: revocación y ciclo de vida

Una sesión de servidor se revoca eliminando o invalidando su registro y consultando ese estado en cada solicitud, de modo que el cierre de sesión, el cambio de contraseña o el cambio de rol surten efecto de inmediato [1]. En un JWT autocontenido, en cambio, la expiración limita la ventana de abuso pero no cancela una copia ya robada (Fig. 1): hasta el instante *exp*, cualquier servidor que valide la firma lo aceptará [3], [4]. Para reducir esa ventana pueden combinarse tokens de acceso de vida corta, tokens de refresco rotatorios con detección de reutilización [9], listas de revocación o una versión de sesión consultada por el servidor. Todas estas medidas reintroducen estado y complejidad: «sin estado» es una propiedad arquitectónica parcial, no una garantía de seguridad.

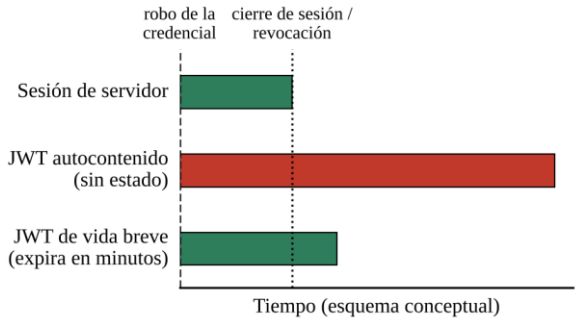


Fig. 1. Ventana de abuso de una credencial robada según el mecanismo (esquema conceptual).

La rotación de tokens de refresco merece atención particular. En cada renovación se emite un par nuevo y el anterior se invalida; si un token ya usado vuelve a presentarse, el servidor interpreta que fue copiado, revoca toda la familia de tokens y obliga a autenticarse de nuevo [9]. El mecanismo mejora la detección de robo, pero requiere almacenar el estado de cada familia, de modo que el emisor deja de ser realmente «sin estado». La amenaza A7 afecta sobre todo a las sesiones de servidor: la defensa es regenerar el identificador al autenticar y al cambiar de privilegio, cosa

que el JWT evita casi por construcción porque se emite un token nuevo en cada autenticación [1].

B. Seguridad: XSS, CSRF y almacenamiento

La Tabla II cruza las variantes de almacenamiento con las amenazas A1, A2 y A3. Un JWT en *localStorage* queda directamente expuesto a XSS y no tiene protección contra la persistencia de la copia robada [7]. Un JWT enviado en cabecera *Authorization* no se adjunta automáticamente y, por tanto, no sufre CSRF clásico, pero obliga a que el script lo conozca, es decir, a almacenarlo donde JavaScript pueda leerlo. La cookie *HttpOnly* elimina esa lectura, aunque exige defensas CSRF: *SameSite* reduce el envío entre sitios, pero OWASP recomienda mantener además tokens anti-CSRF u otras verificaciones, porque *SameSite* no cubre todos los escenarios, como solicitudes entre subdominios del mismo sitio [8], [6]. Obsérvese que el formato (JWT u opaco) es ortogonal al transporte: un JWT dentro de una cookie *HttpOnly* hereda sus ventajas y sus riesgos CSRF.

TABLA II
VARIANTES DE ALMACENAMIENTO FRENTE A LAS AMENAZAS

Variante	A1 XSS lee	A2 CSRF	A3 copia
JWT en *localStorage*	Sí	No	Persiste
JWT en memoria (cabecera)	Parcial	No	Hasta recargar
JWT en cookie *HttpOnly*	No	Sí	Hasta *exp*
Sesión opaca en cookie *HttpOnly*	No	Sí	Hasta invalidar

Ninguna variante neutraliza XSS: cualquier credencial que el navegador adjunte automáticamente puede ser usada por el script inyectado. La defensa primaria sigue siendo evitar la inyección mediante codificación de salida, validación y políticas de contenido [7]. El almacenamiento solo determina cuánto daño adicional se produce: robo y reutilización fuera del navegador (*localStorage*) frente a abuso limitado a la sesión abierta de la víctima (cookie *HttpOnly*).

C. Rendimiento y escalabilidad

La principal ventaja de rendimiento del JWT es la validación local: cualquier instancia o servicio con la clave de verificación puede aceptar el token sin consultar un almacén, lo que facilita el escalado horizontal y reduce la dependencia de un componente compartido [3]. La sesión de servidor exige una consulta por solicitud y, por tanto, un almacén disponible y escalable (base de datos o caché compartida), que pasa a ser un punto único de falla si no se replica. Sin embargo, la ventaja del JWT se erosiona cuando se necesita revocación inmediata, porque la lista de revocación o la versión de sesión vuelven a introducir una consulta por solicitud. Además, un JWT con muchas afirmaciones aumenta el tamaño de cada solicitud y, si viaja en cookie, puede acercarse al límite de 4096 octetos de nombre y valor [6]. Este trabajo no midió latencias; la diferencia real depende del almacén, la red y la carga, y se propone como trabajo futuro.

D. Complejidad, costo y mantenibilidad

La sesión de servidor es conceptualmente simple y está soportada de forma integrada por la mayoría de los marcos de desarrollo web empresariales, lo que reduce el código propio sujeto a errores; su costo es operar el almacén de sesiones y definir expiraciones y limpieza. El JWT desplaza la complejidad hacia la gestión de claves (rotación, distribución de claves públicas), la política de validación, el diseño de expiración y renovación, y la revocación. La existencia de un documento de buenas prácticas dedicado a corregir errores de implementación [4] y de requisitos de verificación específicos para tokens autocontenidos [12] evidencia que estos errores son frecuentes. En mantenibilidad, los *claims* de permisos incluidos en el token pueden quedar desactualizados respecto del estado real del usuario, por lo que el servidor debe consultar los permisos vigentes de todos modos.

E. Síntesis

Las sesiones de servidor ofrecen mejor control de revocación, menor superficie de errores criptográficos y menor complejidad cuando existe un backend único; los JWT aportan valor cuando varios servicios independientes deben validar la identidad sin un almacén común. Un diseño mixto es válido: sesión de servidor en el borde de la aplicación y tokens de vida corta entre servicios internos. Lo importante es decidir con base en la arquitectura y la necesidad de revocación, no en la moda tecnológica. La Tabla IV traduce el análisis en una recomendación por escenario; las filas son una síntesis del autor derivada de los criterios C1 a C5 y de las fuentes citadas, no una afirmación de las fuentes.

TABLA IV
RECOMENDACIÓN SEGÚN EL ESCENARIO ARQUITECTÓNICO

Escenario	Recomendación	Razón principal
Backend único, frontend en el mismo sitio	Sesión de servidor + cookie	Revocación inmediata; *SameSite* viable.
Aplicación de página única con API propia en el mismo sitio	Sesión de servidor + cookie	Evita guardar tokens en *localStorage*.
API consumida por terceros o clientes móviles	JWT breve + refresco rotatorio	Cliente fuera del navegador; validación sin cookies.
Varios servicios internos independientes	Sesión en el borde; JWT breve interno	Validación local sin almacén común.
Operaciones de alto riesgo	Reautenticación + token vinculado a clave	Reduce el valor de una copia robada [11].

IV. APLICACIÓN AL DOMINIO PROPIO

A. Sistema y supuestos

El sistema que se construye en el laboratorio es una aplicación web empresarial en la que usuarios con distintos roles ejecutan procesos de negocio sujetos a reglas y validaciones, por ejemplo, registrar un elemento, cambiar su estado, aprobarlo o anularlo. Para SisGanado se asume un backend único que expone la API y sirve el frontend bajo el mismo sitio, con base de datos relacional y sin consumidores externos que validen credenciales por su cuenta.

B. Decisión

Se decide aplicar sesión administrada en el servidor, con cookie *HttpOnly*, *Secure* y *SameSite*, y no adoptar JWT autocontenido. Las razones, ligadas a los criterios del análisis, son las siguientes. Primero, en C1 el sistema necesita que el cambio de rol, la anulación de un usuario o el cierre de sesión surtan efecto de inmediato, algo que la sesión de servidor ofrece de forma nativa y el JWT solo con estado adicional. Segundo, en C2 la ventaja de validación local del JWT no se aprovecha porque no existen servicios

independientes que verifiquen tokens. Tercero, al servirse frontend y API desde el mismo sitio, *SameSite* y los tokens anti-CSRF son viables y mantienen la credencial fuera del alcance de JavaScript. Cuarto, en C3 y C5 se evita gestionar claves de firma y política de validación de tokens, un área con errores de implementación documentados [4].

Qué cambiaría en la arquitectura: un componente de sesión que regenere el identificador al autenticar y cuando cambian los privilegios; un almacén de sesiones con vencimiento por inactividad y absoluto; la cookie con prefijo *__Host-*, *Secure*, *HttpOnly* y *SameSite* explícito; un mecanismo anti-CSRF en las solicitudes que modifican estado; y una política CORS restrictiva. **Costo:** operar y respaldar el almacén de sesiones y aceptar una consulta por solicitud. **Beneficio:** revocación inmediata, menos código criptográfico propio y un modelo de amenazas más simple de verificar contra el estándar ASVS [12].

Independientemente del mecanismo, la autenticación es necesaria pero no suficiente: la autorización debe combinar identidad, permisos vigentes, ámbito del dato, estado del proceso y parámetros de negocio, y evaluarse siempre en el backend. Por ejemplo, para que un registro avance de «pendiente» a «aprobado», el backend debe comprobar que el usuario tiene permiso de aprobación, que el registro pertenece al ámbito que puede gestionar, que el proceso está en estado «pendiente», que la transición está permitida y que no hay una condición de bloqueo. No basta con que la sesión siga activa o que el JWT tenga una firma válida. Antes de una operación sensible se evalúan conjuntamente: identidad autenticada; firma y algoritmo permitido, si se usa JWT; emisor y audiencia; vigencia (*exp*, *nbf*); estado de sesión o revocación; permisos y roles actuales; pertenencia al recurso; estado del proceso y validez de la transición; nivel de riesgo; y protección CSRF cuando se usan cookies. Si falla cualquier condición obligatoria, la política es denegar por defecto y registrar el evento sin guardar tokens, cookies ni secretos completos. La Tabla III concreta las reglas por proceso.

TABLA III
REGLAS DE SEGURIDAD POR PROCESO PRINCIPAL

Proceso	Controles obligatorios
Inicio de sesión	Validar credenciales y limitar intentos; regenerar el identificador tras autenticar (evita fijación); registrar el evento sin datos sensibles.
Cada solicitud	Validar credencial y vigencia, estado de revocación, permisos actuales y acceso al recurso en la etapa vigente del proceso; no confiar solo en roles o *claims* antiguos.
Cambio de contraseña o sospecha	Invalidar sesiones y tokens de refresco relacionados; exigir nueva autenticación; en JWT, definir cómo se revoca antes de *exp*.
Operaciones críticas	Recomprobar permisos, propiedad, estado del proceso y riesgo en el servidor; verificación adicional si el impacto es alto; el frontend no decide.
Cookies	HTTPS, *Secure*, *HttpOnly*, *SameSite* explícito, dominio y ruta mínimos, defensa CSRF; *HttpOnly* no impide que un script actúe en nombre del usuario.
JWT (si se adopta)	Lista de algoritmos, validación de firma, emisor, audiencia y tiempos; claves protegidas; expiración breve; no registrar tokens completos.

C. Condiciones que cambiarían la decisión y riesgos residuales

La decisión cambiaría si aparecieran clientes móviles o servicios independientes que deban validar la identidad sin consultar el almacén central. En ese caso se usarían tokens de acceso de vida breve, tokens de refresco rotatorios con detección de reutilización, un mecanismo explícito de revocación y, si el riesgo lo justifica, tokens vinculados a clave mediante DPoP [9], [11]. Entre los riesgos residuales de la decisión adoptada están la disponibilidad del almacén de sesiones, que se mitiga con replicación y expiración razonable, y la persistencia de XSS, que se mitiga con codificación de salida, políticas de contenido y reautenticación en operaciones críticas [7]. Esta recomendación se basa en supuestos sobre la arquitectura y debe revisarse si estos cambian.

D. Plan de verificación

Para comprobar que la decisión cumple lo que se afirma, se proponen pruebas con criterio de aceptación explícito: (1) tras cerrar sesión, reutilizar la cookie copiada debe devolver rechazo (401) de inmediato; (2) tras cambiar el rol de un usuario, la siguiente solicitud debe aplicar los permisos nuevos sin volver a iniciar sesión; (3) una solicitud que modifique estado enviada desde otro origen, sin el token anti-CSRF, debe ser rechazada; (4) el identificador de sesión debe cambiar después de autenticar; (5) las cookies de autenticación deben incluir los atributos *Secure*, *HttpOnly* y *SameSite*, lo que puede revisarse en las herramientas del navegador; y (6) los registros no deben contener identificadores completos de sesión. Estas pruebas se alinean con los requisitos de gestión de sesiones del estándar ASVS [12] y pueden ejecutarse en el laboratorio como parte del trabajo futuro.

V. CONCLUSIONES

Hallazgos. (1) El JWT no es más seguro que una sesión de servidor: su principal ventaja es la validación local y su principal costo es la revocación, que obliga a reintroducir estado. (2) Guardar credenciales en *localStorage* amplía el impacto de XSS y no es recomendable; una cookie *HttpOnly* con *Secure* y *SameSite* reduce la lectura directa, pero exige defensas CSRF. (3) Ninguna combinación elimina XSS; el almacenamiento solo limita el daño adicional. (4) Para una aplicación web empresarial con backend único y reglas de negocio, la sesión de servidor es la opción predeterminada; el JWT se justifica solo cuando una necesidad arquitectónica concreta compense su complejidad. (5) La autorización debe evaluar múltiples parámetros en cada operación y no basarse en un único estado como «token válido».

Límites. El análisis es documental: no se midió el rendimiento de ambas opciones, no se realizaron pruebas de ataque controladas ni se evaluaron proveedores de identidad externos, OpenID Connect o *passkeys*. La aplicación al dominio propio se apoya en supuestos sobre la arquitectura del sistema. **Trabajo futuro.** Implementar ambos diseños en el laboratorio y someterlos a escenarios controlados de robo de token, cierre de sesión, cambio de permisos y solicitudes CSRF; medir la latencia de la consulta de sesión frente a la verificación de firma; y evaluar el patrón de backend para frontend y DPoP en un escenario con varios servicios.

REFERENCIAS

[1] OWASP Foundation, «Session Management Cheat Sheet», OWASP Cheat Sheet Series. [En línea]. Disponible en:

https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html. [Accedido: 8-oct-2026].

[2] D. Temoshok, J. Fenton, Y.-Y. Choong, N. Lefkovitz, A. Regenscheid, R. Galluzzo y J. Richer, «Digital Identity Guidelines: Authentication and Authenticator Management», NIST SP 800-63B-4, National Institute of Standards and Technology, Gaithersburg, MD, EE. UU., jul. 2025. doi: 10.6028/NIST.SP.800-63b-4.

[3] M. Jones, J. Bradley y N. Sakimura, «JSON Web Token (JWT)», RFC 7519, Internet Engineering Task Force, mayo 2015. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc7519.

[4] Y. Sheffer, D. Hardt y M. Jones, «JSON Web Token Best Current Practices», RFC 8725, Internet Engineering Task Force, feb. 2020. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc8725.

[5] A. Barth, «HTTP State Management Mechanism», RFC 6265, Internet Engineering Task Force, abr. 2011. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc6265.

[6] S. Bingler, M. West y J. Wilander, «Cookies: HTTP State Management Mechanism», Internet-Draft draft-ietf-httpbis-rfc6265bis-22, Internet Engineering Task Force, en proceso de publicación. [En línea]. Disponible en: https://datatracker.ietf.org/doc/draft-ietf-httpbis-rfc6265bis/. [Accedido: 11-oct-2026].

[7] OWASP Foundation, «Cross Site Scripting Prevention Cheat Sheet», OWASP Cheat Sheet Series. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. [Accedido: 8-oct-2026].

[8] OWASP Foundation, «Cross-Site Request Forgery Prevention Cheat Sheet», OWASP Cheat Sheet Series. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html. [Accedido: 8-oct-2026].

[9] T. Lodderstedt, J. Bradley, A. Labunets y D. Fett, «Best Current Practice for OAuth 2.0 Security», RFC 9700, Internet Engineering Task Force, ene. 2025. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc9700.

[10] OWASP Foundation, «OAuth 2.0 Cheat Sheet», OWASP Cheat Sheet Series. [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/OAuth2_Cheat_Sheet.html. [Accedido: 8-oct-2026].

[11] D. Fett, B. Campbell, J. Bradley, T. Lodderstedt, M. Jones y D. Waite, «OAuth 2.0 Demonstrating Proof of Possession (DPoP)», RFC 9449, Internet Engineering Task Force, sep. 2023. [En línea]. Disponible en: https://www.rfc-editor.org/rfc/rfc9449.

[12] OWASP Foundation, «OWASP Application Security Verification Standard (ASVS) 5.0.0», may. 2025. [En línea]. Disponible en: https://github.com/OWASP/ASVS. [Accedido: 11-oct-2026].